

Inverse Kinematics with Neural Network

Arizona State University

Rumitkumar Patel – 1219783279
Durgesh Reddiyar – 1219611588
Priyank Patel – 1220011416

MAE 547 – Project 2 – Presentation – Fall 2021

Contents

- Problem Statement
- Approach
- Results
- Discussion
- Descriptions
- Contribution

Copyright © 2021 Arizona Board of Regents

Problem Statement

- **Solving Inverse Kinematics with Neural Network and comparing with Robotics Toolbox.**

- Inverse Kinematics is a nonlinear mapping problem. Most real-world applications of robots use inverse kinematics, which increases the need of efficient method to tackle this problem.

- For this project we have used a RRR and RPP robotic arms. We computed their joint space parameters using inverse kinematics (ikine function) from robotics toolbox and neural network.

- Neural networks have been increasingly deployed in the field of robotics.

- Generally, these networks are very accurate in predicting solutions to the non-linear system of equations.
- Conventional methods are less accurate with complex time and space complexity, often taking longer duration to find joint space parameters for complex robots.

Copyright © 2021 Arizona Board of Regents

Approach 'ikine function' based solver

- We begin by taking robot end effecter pose as input from the user. This input condition is fed to the 'ikine' solver.
- We constrained the solver to run for a maximum of 1000 iterations. This does limit the computation of joint space parameters for complex end-effector pose. However, it is sufficient to show that our method works.
- We use these joint parameters to simulate the RRR and the RPP robotic arms using the animate function.

Copyright © 2021 Arizona Board of Regents

Approach- Neural Network model

- For the neural network model-based approach we first begin by training the model.
- The dataset was obtained using 'ikine' function. By providing 10000 random input end-effector points and storing the joint space parameters.
- Once the trained model was obtained, we took the end-effector position as input parameter from the user and used the trained model to compute the joint space parameters.
- Next, we simulated the RRR and RPP robots using the predicted joint space parameters.

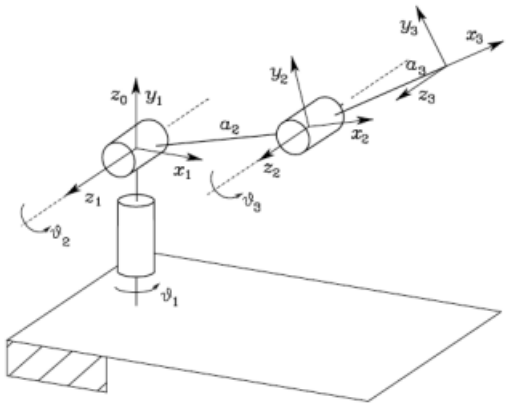
Copyright © 2021 Arizona Board of Regents

Discussion

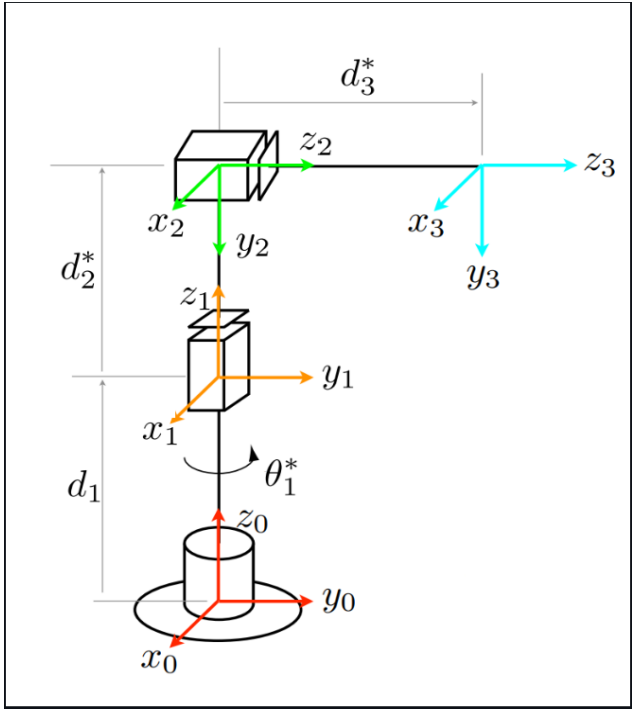
- The robotic arms contains two links and three revolute joints for the RRR arm and two links and one revolute joint and two prismatic joints for the RPP arm.
- While calculating the joint space parameters there were many end-effector poses which could not be computed, this could be attributed to two reasons.
- One is that the iteration limit of 'ikine' solver set to 1000 could not compute the joint space parameters due to timeout.
- Another reason, a more traditional one, is that the input end-effector pose could be a singularity condition.
- While obtaining our dataset, we did not include the condition which could be placed under the above mentioned two error causing conditions. We categorize this as a data pre-processing step.
- For the simulation of the robotic arm, we used the animate() function from the robotic toolbox, which proved to be very intuitive to work on.

Copyright © 2021 Arizona Board of Regents

Figure of the robotic arms



1. RRR Robotic Arm



2. RPP Robotic Arm

Copyright © 2021 Arizona Board of Regents

Results

- Since we have two different models solving the inverse kinematics to obtain the joint space parameters. We have decided to compare both the simulations and observe the differences in the results obtained.

- The neural network model was trained for 20000 epochs, with the train test split of 80:20. The size of our dataset was 10000 entries.

- The model achieved the accuracy of ~ 75 % on the test dataset.

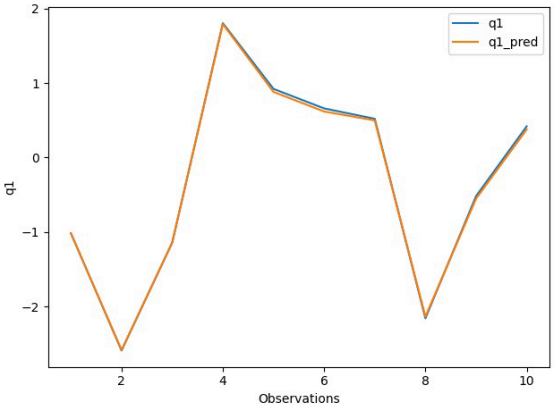
- For the RRR robot we gave the input parameter [1 4 9] and simulated the robot using the traditional 'ikine' solver and neural network-based solver. The model and the traditional solver returned similar output joint space parameters which are [78.7 deg, 4.3 deg, 120.1 deg].

- The above-mentioned joint space parameters simulated almost identical results with difference ~1deg.

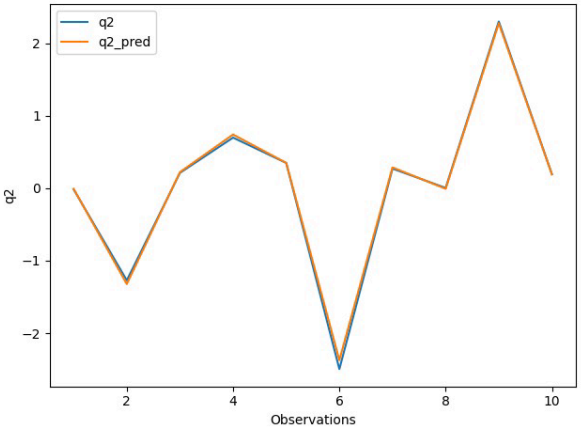
- These results computed by the neural network model are highly accurate.

Copyright © 2021 Arizona Board of Regents

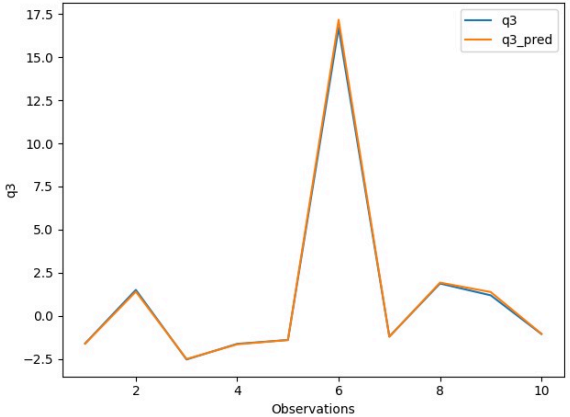
Comparison of results



1. Comparison of q1 between ikine and NN



2. Comparison of q2 between ikine and NN



1. Comparison of q3 between ikine and NN

- The comparison of results using robotics toolbox and neural network of solution is provided in the above graphs. For q_i, where $i = 1,2,3$ and q_{i_pred}, where $i = 1,2,3$. Here, q_i are the solutions obtained using robotics toolbox and q_{i_pred} are the solutions obtained using neural network.
- As we can deduce from above graphs, that neural networks are very reliable for getting an inverse kinematics solutions and efficient in time and computation as well. And we can observe that there is not much of a divergence in the plots of q_i and q_{i_pred}.

Copyright © 2021 Arizona Board of Regents

Description of the codebase

- gen.m and gen_rpp.m files is used to generate data for training the neural network
- train.py file is used to train the neural network.
- inference.py file is used to do custom predictions.
- Model folder contains trained models. Dataset folder contains temporary files needed to transfer data from python and MATLAB.
- rrr_3_link_toolbox.py script calculates inverse kinematics with robotics toolbox for 3 revolute manipulator.
- rrr_3_link_NN.py this script is used calculate inverse kinematics with neural networks for 3 revolute manipulators.
- rpp_toolbox.py script calculates inverse kinematics with robotics toolbox for 1 revolute and 2 prismatic manipulator
- rpp_NN.py this script is used calculate inverse kinematics with neural networks for 1 revolute and 2 prismatic manipulators.

Copyright © 2021 Arizona Board of Regents

Instructions on how to run the code

- Required IDE's : MATLAB and any python development environment(which can be accessed with command line).

- Please extract all the files and folders from the .zip file that has been uploaded:
- Open **run_project** folder for running the project.

Procedure for RRR Robot Inverse Kinematics with Neural network and Robotics toolbox.

- Open the **file rrr_3_link_toolbox.m** which will ask for end effector pose in the form [x y z]. After running this script, you can see 3D animation of RRR robot from initial pose to the proved end effector pose.

- Open the file **rrr_3_link_NN.m** which calls py_rrr.py and installs required packages and libraries required by neural network backend. After running this script, you can see 3D animation of RRR robot from initial pose to the proved end effector pose.

Copyright © 2021 Arizona Board of Regents

Instructions on how to run the code (cont.)

Procedure for RPP Robot Inverse Kinematics with Neural network and Robotics toolbox.

- Open the file rpp_toolbox.m which will ask for end effector pose in the form [x y z]. After running this script, you can see 3D animation of RPP robot from initial pose to the proved end effector pose.

- Open the file rpp_NN.m which calls py_rrr.py and installs required packages and libraries required by neural network backend. Also, will ask for end effector pose in the form [x y z]. After running this script, you can see 3D animation of RPP robot from initial pose to the proved end effector pose.

Copyright © 2021 Arizona Board of Regents

Contribution

Contribution of each team member:

- All team members contributed to project equally and sincerely.
- My team members are multidisciplinary, so it was great to have different expertise to tackle the problem it was a great experience to work with amazing teammates.

Durgesh	33.34%	Idea formation	Brain storming	Neural network training	Testing whole system	Literature Review	Results extraction	Presentation
Priyank	33.33%	Idea formation	Brain storming	MATLAB scripting	Testing neural network	Literature Review	Results Video	Presentation
Rumitkumar	33.33%	Team formation	Brain storming	Data generation	Testing MATLAB scripts	Literature Review	Results verification	Presentation

Durgesh Reddiyar Priyank Patel Rumitkumar Patel

Copyright © 2021 Arizona Board of Regents

THANK YOU

dreddiya@asu.edu
linkedin.com/in/durgeshreddiyar/

rpatel71@asu.edu
linkedin.com/in/rumitkumar-patel-5b6a34144/

ppatel65@asu.edu
https://www.linkedin.com/in/priyankpatel1207/

